

# Thread-Zero Truth Model Specification

*Canonical Reference for Event Integrity, Hash Chain Rules, and Audit Reconstruction*

**Author:** leon **Date:** 2026-07-09

## Table of Contents

1. **Purpose & Scope** Goals and coverage of the Truth Model
2. **Event Model** 2.1 — Event Definition & Schema 2.2 — Event Type Registry 2.3 — Sequence Number Rules 2.4 — Payload Integrity
3. **Hash Chain Rules** 3.1 — Overview 3.2 — Hash Computation 3.3 — Chain Linking 3.4 — Validation Algorithm 3.5 — Forking
4. **Truth Anchor** 4.1 — Definition 4.2 — Anchor Schema 4.3 — Anchor Frequency Rules 4.4 — Merkle Root Construction 4.5 — Anchor Authority 4.6 — External Anchoring (Optional)
5. **Reconstruction Rules** 5.1 — Principle of Reconstruction 5.2 — Reconstruction Procedure 5.3 — Partial Reconstruction 5.4 — Conflict Resolution 5.5 — Reconstruction Guarantees
6. **Audit Export Format** 6.1 — Overview 6.2 — Export Bundle Structure 6.3 — Manifest Schema 6.4 — JSONL Event File Format 6.5 — Audit Export Validation Checklist
7. **Examples** 7.1 — Genesis Event 7.2 — Assertion Event 7.3 — Truth Anchor 7.4 — Tamper Detection Scenario 7.5 — Audit Export Manifest
8. **Appendix A** — Glossary
9. **Appendix B** — Cryptographic Algorithm Summary
10. **Appendix C** — Changelog

# 1. Purpose & Scope

This document defines the **ThreadZero Truth Model (TTM)**, the authoritative specification for how ThreadZero records, chains, anchors, and audits events. The TTM is the *truth layer* — it governs immutability, tamper-evidence, and forensic reconstructibility across all event streams.

This specification is the single canonical reference for implementors, auditors, and integration partners. All ThreadZero components that produce, consume, or relay events **MUST** conform to the rules herein. Any deviation from this specification constitutes a non-conformance that may compromise chain integrity.

## Goals

- Provide a deterministic, verifiable record of every event
- Enable tamper-detection via hash chaining
- Anchor truth at defined intervals for external verification
- Support complete reconstruction of state from the event log alone
- Define a standard audit export format for compliance and forensics

### ① Scope Note

This specification covers the on-chain event record and its cryptographic integrity mechanisms. It does not govern transport protocols, storage backends, or access control policies, except where those systems interact with event committal or chain validation.

## 2. Event Model

### 2.1 Event Definition

An **Event** is the atomic unit of the ThreadZero record. Every state change, action, or observation must be represented as a discrete Event. Events are **immutable once committed** — no field may be altered, patched, or deleted after the event has been written to the chain. The only permitted response to an incorrect event is the issuance of a corrective subsequent event.

#### Event Schema (JSON):

```
{  "event_id":      "<UUID v4>",  "event_type":    "<string – see §2.2>",
"timestamp":      "<ISO 8601 UTC – e.g. 2026-07-09T23:02:00Z>",
"sequence_no":    "<uint64 – monotonically increasing, no gaps>",  "actor_id":
"<string – identity of the originating agent or user>",  "session_id":
"<UUID v4 – groups events within a logical session>",  "payload":      {
... },  // event-type-specific structured data  "tags":
[ "<string>", ... ],  // optional classifier labels  "prev_hash":      "<SHA-
256 hex of previous event – null for genesis>",  "self_hash":      "<SHA-256
hex of this event (computed after prev_hash is set)>" }
```

### 2.2 Event Type Registry

The following table enumerates all recognized event types. Implementations **MUST NOT** emit unregistered type codes. Extension type codes require a specification amendment.

| Type Code         | Category   | Description                             | Payload Required Fields                     |
|-------------------|------------|-----------------------------------------|---------------------------------------------|
| SYSTEM_BOOT       | Lifecycle  | System or session initialization        | version, node_id                            |
| SESSION_OPEN      | Lifecycle  | New session begins                      | session_id, actor_id                        |
| SESSION_CLOSE     | Lifecycle  | Session ends cleanly                    | session_id, duration_ms                     |
| STATE_CHANGE      | Mutation   | Any mutable state transition            | entity_id, field, old_value, new_value      |
| ASSERTION_MADE    | Truth      | A claim or fact is asserted by an actor | claim_id, content_hash, confidence          |
| ASSERTION_REVOKED | Truth      | A prior assertion is invalidated        | claim_id, reason                            |
| ANCHOR_CREATED    | Integrity  | Truth anchor recorded (see §4)          | anchor_id, anchor_hash, range               |
| AUDIT_REQUEST     | Compliance | External audit export triggered         | requestor_id, export_range                  |
| ERROR_EVENT       | Diagnostic | Non-fatal anomaly logged                | error_code, message                         |
| TAMPER_ALERT      | Security   | Hash chain discontinuity detected       | detected_at_seq, expected_hash, actual_hash |

## 2.3 Sequence Number Rules

- `sequence_no` MUST start at 0 for the genesis event of a chain
- `sequence_no` MUST increment by exactly 1 for each subsequent event
- No gaps are permitted; a gap constitutes evidence of deletion or tampering
- `sequence_no` is scoped to a single chain; parallel chains maintain independent counters

### ⚠ Critical Constraint

A gap in `sequence_no` is not a recoverable warning — it is treated as confirmed tampering and MUST trigger a `TAMPER_ALERT` event immediately upon detection. Do not attempt to fill gaps by synthesizing events.

## 2.4 Payload Integrity

The `payload` field is type-specific structured data. For `ASSERTION_MADE` events, the payload MUST include `content_hash` — the SHA-256 of the raw assertion content — rather than the raw content itself. This design avoids storing potentially sensitive data in the immutable chain while still enabling external verification of the asserted content by any party who independently holds the source material.

## 3. Hash Chain Rules

### 3.1 Overview

ThreadZero uses a **forward-linked SHA-256 hash chain**. Each event commits to the identity of the event immediately preceding it, forming an unbroken cryptographic dependency. Any retroactive alteration of any event — including modification of a single byte in any field — invalidates every subsequent hash linkage and is immediately detectable during chain validation.

## 3.2 Hash Computation

The following procedure MUST be followed exactly. Deviation in serialization order, whitespace handling, or encoding will produce a different hash and render the event invalid.

### Self-hash computation procedure:

```
Step 1: Serialize the event to canonical JSON (keys alphabetically sorted at all nesting levels, no whitespace)
Step 2: Set self_hash = "" (empty string) during computation - the self_hash field must be present but blank
Step 3: Compute: self_hash = SHA-256(canonical_json_string)
Step 4: Replace self_hash in the stored record with the computed hex value
```

### Canonical JSON rules:

- Keys are sorted lexicographically at every nesting level
- No trailing commas
- No whitespace (no spaces, no newlines)
- Unicode strings encoded as UTF-8
- Numbers: no trailing zeros, no leading zeros except for `0.x` values

## 3.3 Chain Linking

- The first event (genesis) sets `prev_hash = null`
- Every subsequent event sets `prev_hash = self_hash` of the immediately preceding event (`sequence_no - 1`)
- An event is valid only if its `prev_hash` exactly matches the stored `self_hash` of the prior event

## 3.4 Validation Algorithm

The following algorithm MUST be applied sequentially for every event in the chain during a validation pass. Validation is a read-only operation and MUST NOT modify any event record.

```
For each event E[n] in the chain (n >= 1):
  1. Verify E[n].sequence_no == E[n-1].sequence_no + 1
     → Failure: sequence gap detected
  2. Verify E[n].prev_hash == E[n-1].self_hash
     → Failure: hash linkage broken
  3. Recompute E[n].self_hash using §3.2 procedure
  4. Verify recomputed hash == stored E[n].self_hash
     → Failure: event content has been altered
  If any verification fails:
    → Emit TAMPER_ALERT event (do not modify chain)
    → Halt forward traversal
    → Escalate to nearest preceding Truth Anchor (see §4)
```

## 3.5 Forking

Chain forking is **not permitted** in a single ThreadZero instance. A fork condition is defined as two or more events sharing the same `prev_hash` value (i.e., two events both claiming to be the successor of the same parent). Upon detection:

- The later-timestamped branch is immediately marked `INVALIDATED`
- Dispute resolution escalates to the nearest preceding Truth Anchor (see §4)
- A `TAMPER_ALERT` event is appended to the authoritative branch

## 4. Truth Anchor

### 4.1 Definition

A **Truth Anchor** is a cryptographically signed checkpoint that freezes the hash chain state at a specific sequence number range. Anchors serve as trusted reference points for reconstruction, external verification, and dispute resolution. They do not interrupt the event chain — they are appended to it as `ANCHOR_CREATED` events and also maintained in a dedicated anchor registry.

### 4.2 Anchor Schema

```
{  "anchor_id":      "<UUID v4>",  "anchor_version":  "1.0",
"created_at":      "<ISO 8601 UTC>",  "chain_id":        "<UUID v4 -
identifies the event chain>",  "seq_from":          <uint64 - first
sequence_no in anchored range>,  "seq_to":              <uint64 - last
sequence_no in anchored range>,  "root_hash":           "<Merkle root of all
self_hashes in [seq_from, seq_to]>",  "anchor_hash":      "<SHA-256 of
canonical anchor JSON (excluding anchor_hash itself)>",  "signature":
"<Ed25519 signature of anchor_hash using anchor authority key>",
"authority_key_id": "<identifier of the signing key>",  "external_ref":
"<optional - external timestamp authority or blockchain tx ID>" }
```

## 4.3 Anchor Frequency Rules

| Trigger Condition                        | Requirement                                           |
|------------------------------------------|-------------------------------------------------------|
| Every 10,000 events elapsed              | Anchor MUST be created (mandatory maximum interval)   |
| <code>SESSION_CLOSE</code> event emitted | Anchor MUST be created before the session closes      |
| <code>AUDIT_REQUEST</code> event emitted | Anchor MUST be created before the export is generated |
| Any authorized actor requests            | Anchor MAY be created at any time                     |

### **i** Note

Anchors themselves are appended to the event chain as `ANCHOR_CREATED` events. The anchor event's payload includes the `anchor_id`, `anchor_hash`, and the covered range (`seq_from` to `seq_to`).

## 4.4 Merkle Root Construction

The `root_hash` field is the apex of a binary Merkle tree built from the `self_hash` values of all events in the range `[seq_from, seq_to]`, ordered strictly by `sequence_no`.

```
Construction rules:  Leaf nodes:  SHA-256(self_hash_of_event[n]) for
each event,          ordered by sequence_no ascending      Internal
nodes: SHA-256( left_child_hash || right_child_hash )      where
|| denotes binary concatenation  Odd leaf count: The last leaf node is
duplicated to form a pair      Root:          The single hash at the apex of
the completed tree
```

## 4.5 Anchor Authority

The anchor authority is a designated Ed25519 key pair held by the ThreadZero root node or a designated trust service. The `authority_key_id` references the corresponding public key in the ThreadZero key registry.

### Verification procedure:

1. Retrieve the public key identified by `authority_key_id` from the key registry
2. Serialize the anchor object to canonical JSON with `anchor_hash` set to ""
3. Recompute `anchor_hash` = SHA-256 of the canonical anchor JSON
4. Verify the Ed25519 `signature` against `anchor_hash` using the retrieved public key
5. If signature verification fails, the anchor is invalid and MUST NOT be used as a reconstruction reference point

## 4.6 External Anchoring (Optional)

For highest-assurance deployments, `anchor_hash` MAY be committed to an external immutable log (e.g., a public blockchain or an RFC 3161 timestamp authority). The resulting transaction ID or signed timestamp token is stored in the `external_ref` field. External anchoring provides a tamper-evident reference that exists independently of the ThreadZero system itself, enabling third-party verification without access to internal infrastructure.



## 5. Reconstruction Rules

### 5.1 Principle of Reconstruction

The ThreadZero event log is the **single source of truth**. Any system state at any point in time **MUST** be fully derivable by replaying the event log from genesis (or from the nearest valid Truth Anchor) forward to the desired `sequence_no`. No external state store, cache, or snapshot is authoritative unless it can be traced back to and verified against the event log.

### 5.2 Reconstruction Procedure

**Step 1 — Locate anchor:** Find the nearest `ANCHOR_CREATED` event at or before the target sequence number. Verify the anchor's `root_hash` and Ed25519 `signature` per §4.5.

**Step 2 — Validate forward chain:** Beginning at `seq_from` of the verified anchor, apply the validation algorithm (§3.4) for each event forward up to the target `sequence_no`. Halt on any validation failure.

**Step 3 — Replay mutations:** Apply `STATE_CHANGE` events in strict sequence order to derive entity state. Apply `ASSERTION_MADE` and `ASSERTION_REVOKED` events to build and update the assertion truth table.

**Step 4 — Verify terminal state:** If reconstruction targets the current tip of the chain, verify that the derived state matches the live system state. Any discrepancy constitutes a `TAMPER_ALERT` condition.

### 5.3 Partial Reconstruction

If the genesis event is unavailable, reconstruction may begin from any valid Truth Anchor. System state prior to that anchor cannot be reconstructed from the log alone — it requires an external snapshot explicitly tagged with the corresponding `anchor_id`. Partial reconstructions **MUST** be clearly marked as such in any audit or compliance report.

## 5.4 Conflict Resolution During Reconstruction

| Conflict Type                                                                                          | Resolution Rule                                                              |
|--------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------|
| Two <code>STATE_CHANGE</code> events target same <code>(entity_id, field)</code> with equal timestamps | The event with the higher <code>sequence_no</code> wins                      |
| <code>ASSERTION_REVOKED</code> and <code>ASSERTION_MADE</code> for same <code>claim_id</code>          | <code>ASSERTION_REVOKED</code> always supersedes, regardless of replay order |
| <code>TAMPER_ALERT</code> events encountered during replay                                             | Do not mutate state — treat as diagnostic markers only                       |

## 5.5 Reconstruction Guarantees

Given a valid, unbroken chain from genesis (or a valid anchor) to the target tip, the ThreadZero Truth Model guarantees the following properties:

| Property            | Guarantee                                                                               |
|---------------------|-----------------------------------------------------------------------------------------|
| <b>Determinism</b>  | The same event log always produces exactly the same derived state                       |
| <b>Completeness</b> | No state exists that is not explained by at least one event in the log                  |
| <b>Auditability</b> | Every state transition has a traceable <code>actor_id</code> and <code>timestamp</code> |

## 6. Audit Export Format

### 6.1 Overview

The **Audit Export** is a self-contained, human-readable and machine-parseable bundle used for compliance review, forensic analysis, or external handoff. It encapsulates a contiguous event range, all relevant Truth Anchors covering that range, public keys needed for signature verification, and a manifest that enables end-to-end integrity checking without access to the live ThreadZero system.

## 6.2 Export Bundle Structure

A ThreadZero Audit Export is delivered as a ZIP archive. The archive MUST conform exactly to the following layout. Presence of unlisted files at the root level is permitted but MUST be declared in `manifest.json` under an optional `extra_files` array.

```
threadzero-audit-<export_id>/ |─ manifest.json          # Export
metadata and integrity summary |─ events/ |   |─ events-000001.jsonl      #
Events in sequence, JSONL format (1000 events/file) |   |─ events-
000002.jsonl      # Continue for additional event files as needed |─ anchors/
|   |─ anchors.json          # All Truth Anchors covering the export range
|─ keys/ |   |─ public-keys.json      # Public keys for anchor signature
verification |─ verification/   |─ checksums.sha256          # SHA-256
checksums of all files in the bundle
```

## 6.3 Manifest Schema

```
{  "export_id":      "<UUID v4>",  "export_version":    "1.0",
  "generated_at":    "<ISO 8601 UTC>",  "generated_by":      "<actor_id of
requestor>",  "chain_id":      "<UUID v4>",  "seq_from":      "<uint64>",
  "seq_to":          "<uint64>",  "total_events":    "<uint64>",
  "anchor_count":    "<integer>",  "genesis_hash":    "<self_hash of
sequence_no 0, or null if export is partial>",  "tip_hash":
"<self_hash of last event in export>",  "export_root_hash":  "<Merkle root
of all self_hashes in export range>",  "integrity_status":  "VERIFIED |
PARTIAL | UNVERIFIED",  "notes":      "<optional free text>" }
```

The `integrity_status` field MUST be set by the exporting system at time of generation and MUST reflect the result of a full chain validation pass over the exported range immediately prior to export.

## 6.4 JSONL Event File Format

Each file in the `events/` directory uses JSON Lines format. The rules below are strict and non-negotiable for machine-parseable compliance exports:

- Each line is a complete, self-contained event JSON object as defined in §2.1
- Each line is terminated by a single newline character (`\n`)
- Lines MUST be ordered strictly by `sequence_no`, ascending
- No blank lines are permitted anywhere in the file
- Each file contains exactly 1,000 events, except the final file which may contain fewer
- File numbering is 1-based and zero-padded to 6 digits (e.g., `events-000001.jsonl`)

## 6.5 Audit Export Validation Checklist

Receiving parties and auditors MUST perform the following steps before relying on an export for legal or compliance purposes:

6. Verify `checksums.sha256` against all files present in the bundle
7. Confirm all anchor signatures using the public keys in `keys/public-keys.json`
8. Confirm `manifest.export_root_hash` matches the Merkle root independently computed from all events in the export
9. Walk the event chain per §3.4 from `seq_from` to `seq_to`
10. Confirm no `sequence_no` gaps exist anywhere in the export range
11. Confirm `manifest.tip_hash` equals the `self_hash` of the final event in the export
12. Confirm `integrity_status = "VERIFIED"` before relying on the export for legal or compliance purposes

### Warning

An export with `integrity_status` of `PARTIAL` or `UNVERIFIED` MUST NOT be accepted as authoritative for compliance, legal proceedings, or forensic use without explicit written acknowledgement of the limitation by both parties.

## 7. Examples

### 7.1 Example: Genesis Event

```
{  "event_id":      "00000000-0000-0000-0000-000000000001",  "event_type":
"SYSTEM_BOOT",    "timestamp":  "2026-07-09T23:00:00Z",  "sequence_no": 0,
"actor_id":       "system:threadzero-core",  "session_id":  "a1b2c3d4-e5f6-
7890-abcd-ef1234567890",  "payload": {      "version":  "1.0.0",
"node_id":  "node-alpha-01" },  "tags":      ["genesis", "lifecycle"],
"prev_hash":      null,  "self_hash":
"e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855" }
```

#### **Note**

The `self_hash` shown above is illustrative. Always recompute per §3.2 using the canonical JSON of the actual event. Do not copy hash values from examples into production records.

### 7.2 Example: Assertion Event (sequence\_no 1)

```
{  "event_id":      "11111111-1111-1111-1111-111111111111",  "event_type":
"ASSERTION_MADE",  "timestamp":  "2026-07-09T23:00:05Z",  "sequence_no":
1,  "actor_id":     "user:leon",  "session_id":  "a1b2c3d4-e5f6-7890-abcd-
ef1234567890",  "payload": {      "claim_id":      "claim-0001",
"content_hash":  "abc123def456...",  "confidence":  0.97 },  "tags":
["assertion", "high-confidence"],  "prev_hash":
"e3b0c44298fc1c149afbf4c8996fb924...",  "self_hash":
"7f83b1657ff1fc53b92dc18148ald65..." }
```

### 7.3 Example: Truth Anchor (covering sequence\_no 0–999)

```
{  "anchor_id":      "aaaaaaaa-aaaa-aaaa-aaaa-aaaaaaaaaaaa",
"anchor_version":   "1.0",  "created_at":      "2026-07-09T23:10:00Z",
"chain_id":        "cccccccc-cccc-cccc-cccc-cccccccccccc",  "seq_from":
0,  "seq_to":        999,  "root_hash":
"d4e5f6a7b8c9d0e1f2a3b4c5d6e7f8a9...",  "anchor_hash":
"1a2b3c4d5e6f7890abcdef1234567890...",  "signature":
"ED25519_SIG_BYTES_BASE64_HERE==",  "authority_key_id": "key-threadzero-
root-2026-01",  "external_ref":      null }
```

## 7.4 Example: Tamper Detection Scenario

**Scenario:** An adversary deletes the event at `sequence_no 42` and renumbers events 43 and onward to fill the gap (i.e., original event 43 is relabeled as 42, event 44 as 43, and so on).

**Detection:** During validation of the relabeled event at `sequence_no 42` (originally 43), the validator checks its `prev_hash`. That `prev_hash` was computed against the `self_hash` of the original event 42 — which no longer exists in the chain. The stored `prev_hash` of this event does not match the `self_hash` of the new preceding event (original event 41). The mismatch is detected and a `TAMPER_ALERT` is emitted at `sequence_no 42`.

**Resolution:** Roll back to the nearest valid Truth Anchor before `sequence_no 42`. Declare all events from the divergence point onward as `INVALIDATED` pending investigation. Initiate a forensic reconstruction from the anchor.

## 7.5 Example: Audit Export Manifest

```
{  "export_id":      "fedcba09-8765-4321-fedc-ba0987654321",
  "export_version":  "1.0",    "generated_at":      "2026-07-09T23:59:00Z",
  "generated_by":    "user:leon",  "chain_id":      "cccccccc-cccc-cccc-
cccc-cccccccccccc",  "seq_from":      0,    "seq_to":      4999,
  "total_events":    5000,    "anchor_count":      5,    "genesis_hash":
  "e3b0c44298fc1c149afb4c8996fb924...",    "tip_hash":
  "9f86d081884c7d659a2feaa0c55ad015...",    "export_root_hash":
  "a665a45920422f9d417e4867efdc4fb8...",    "integrity_status":  "VERIFIED",
  "notes":           "Full chain export for Q2 2026 compliance review." }
```

## Appendix A — Glossary

| Term                  | Definition                                                                                                                                      |
|-----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Chain</b>          | An ordered sequence of events linked by SHA-256 hash pointers, forming an immutable and tamper-evident record.                                  |
| <b>Genesis Event</b>  | The first event in a chain. Defined by <code>sequence_no = 0</code> and <code>prev_hash = null</code> .                                         |
| <b>Truth Anchor</b>   | A cryptographically signed checkpoint that freezes and commits to the hash chain state over a defined sequence range.                           |
| <b>Merkle Root</b>    | A single hash that summarizes a set of hashes via a binary tree structure, enabling efficient and tamper-evident verification of any subset.    |
| <b>Tamper Alert</b>   | A diagnostic event ( <code>TAMPER_ALERT</code> ) signaling a detected hash chain discontinuity. Does not mutate state.                          |
| <b>Reconstruction</b> | The process of deriving system state at a given point in time by replaying the event log from genesis or a valid Truth Anchor.                  |
| <b>Audit Export</b>   | A portable, self-verifying ZIP bundle of events and anchors prepared for external compliance review or forensic handoff.                        |
| <b>Canonical JSON</b> | A deterministic JSON serialization format with lexicographically sorted keys, no whitespace, and UTF-8 encoding. Required for hash computation. |

## Appendix B — Cryptographic Algorithm Summary

| Algorithm                 | Usage in TTM                                                                                                           | Notes                                                                                                             |
|---------------------------|------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| <b>SHA-256</b>            | Event <code>self_hash</code> , <code>prev_hash</code> , Merkle tree node hashing, <code>anchor_hash</code> computation | NIST standard; 256-bit (32-byte) output; collision-resistant for current threat models                            |
| <b>Ed25519</b>            | Truth Anchor signature generation and verification ( <code>signature</code> field)                                     | Edwards-curve DSA; fast, compact 64-byte signatures; widely supported in modern cryptographic libraries           |
| <b>Binary Merkle Tree</b> | Anchor <code>root_hash</code> construction over event ranges                                                           | Binary tree with SHA-256 at each node; duplicate last leaf if odd leaf count; ordered by <code>sequence_no</code> |

## Appendix C — Changelog

---

| Version     | Date       | Author | Summary                                                                                                                                                             |
|-------------|------------|--------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>v1.0</b> | 2026-07-09 | leon   | Initial release — ThreadZero Truth Layer Specification. Covers event model, hash chain rules, Truth Anchor protocol, reconstruction rules, and audit export format. |

**ThreadZero Truth Model Specification v1.0 2026-07-09 INTERNAL TECHNICAL REFERENCE**